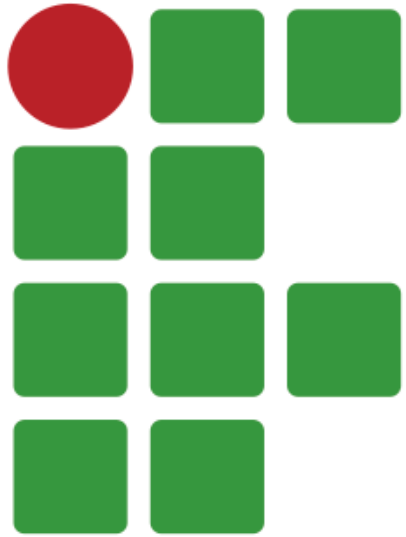


INSTITUTO FEDERAL
Brasília

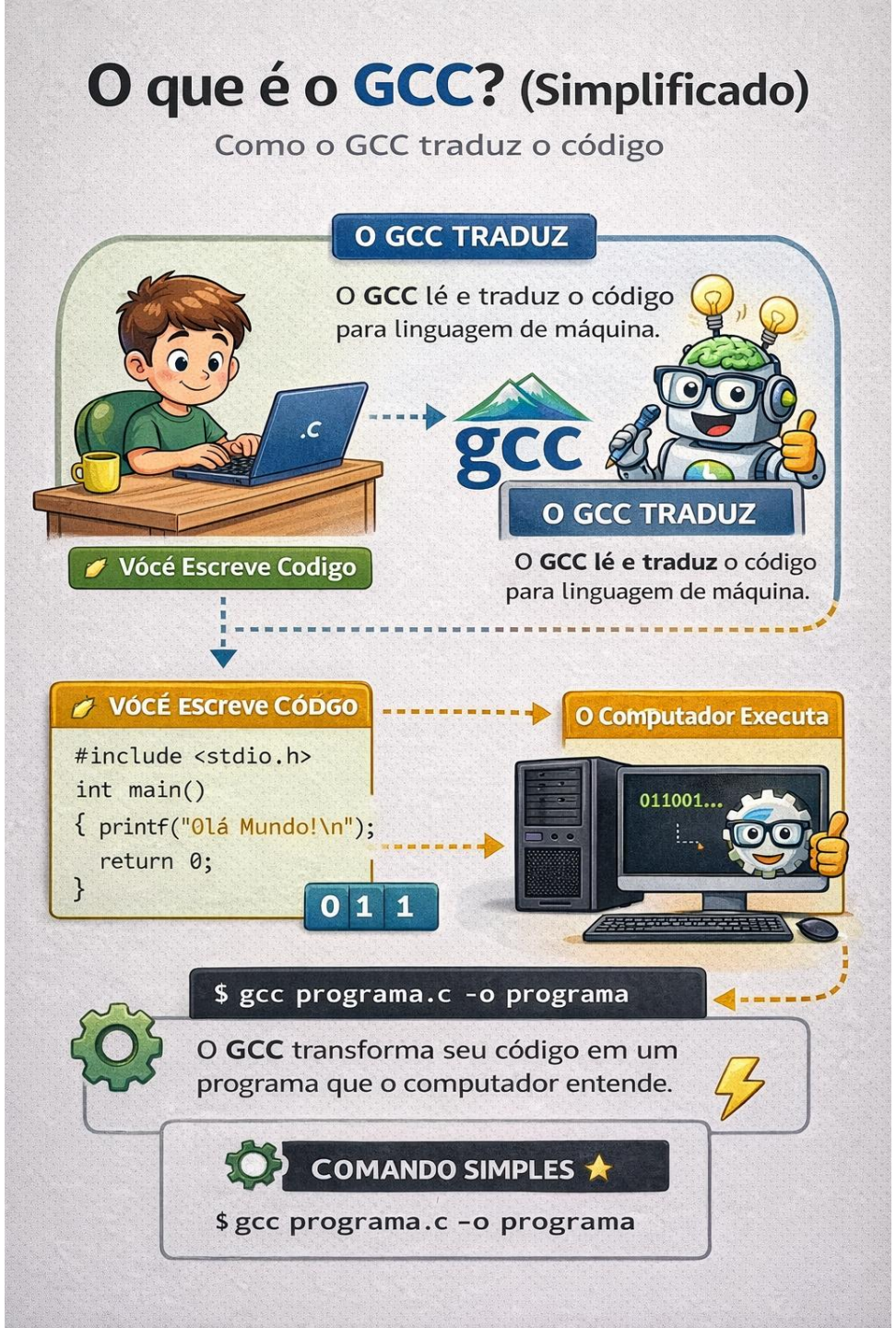
Algoritmos e Programação de Computadores TAI-3S

Prof. MSc. Thiago Neves
IFB Campus Taguatinga
29/03/2026

AULA 02

Configurando o ambiente
Estruturas básicas

GCC (GNU Compiler Collection)



Configurando o UBUNTU

Abra o **Terminal** (atalho: Ctrl + Alt + T) e digite:

$sudo apt update

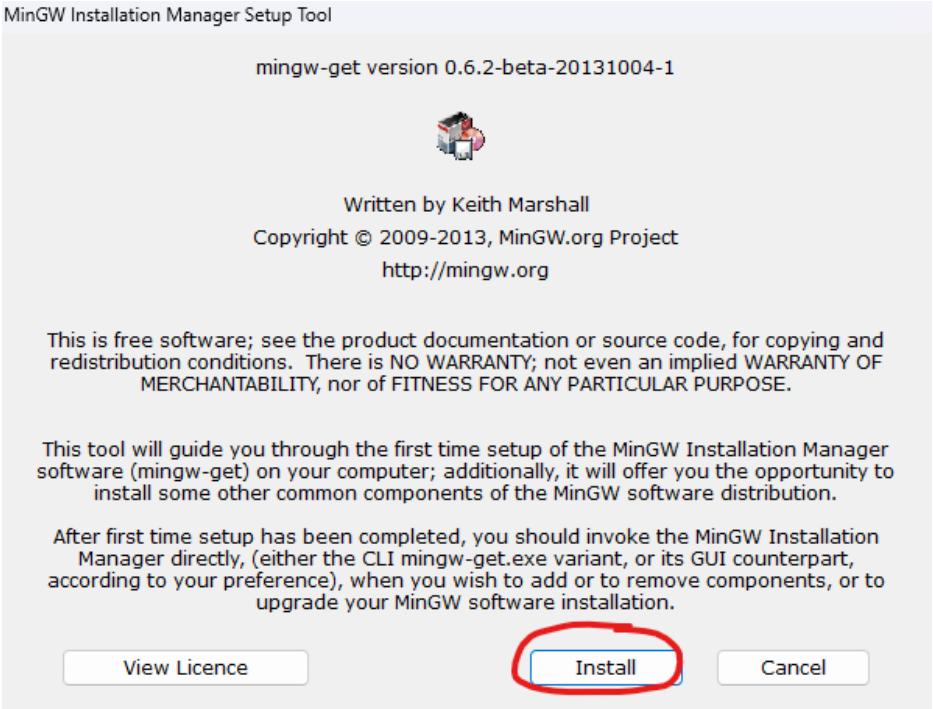
$sudo apt install build-essential

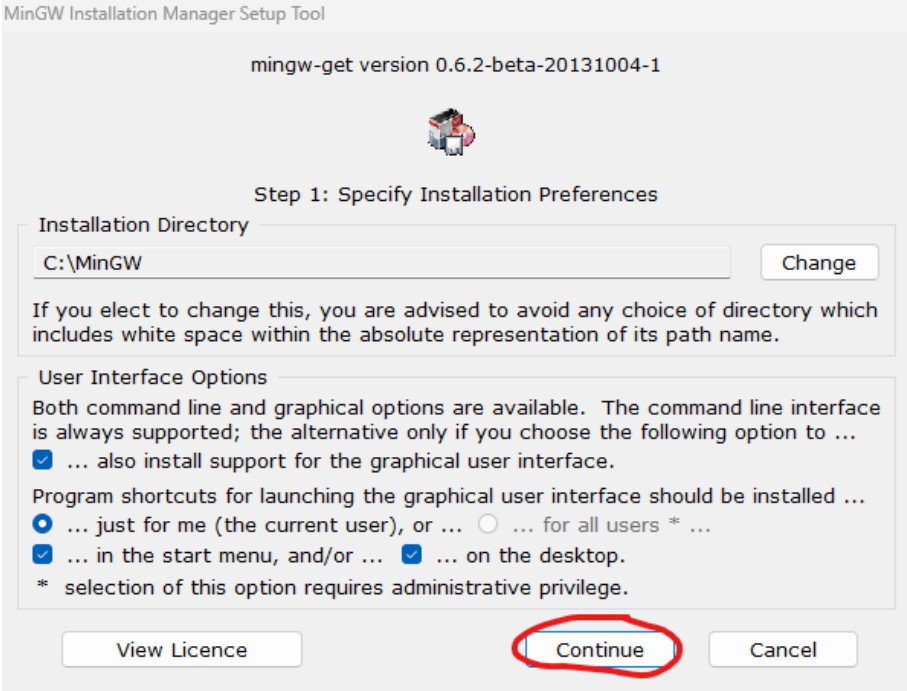
$gcc --version

Configurando o WINDOWS (Parte 1)

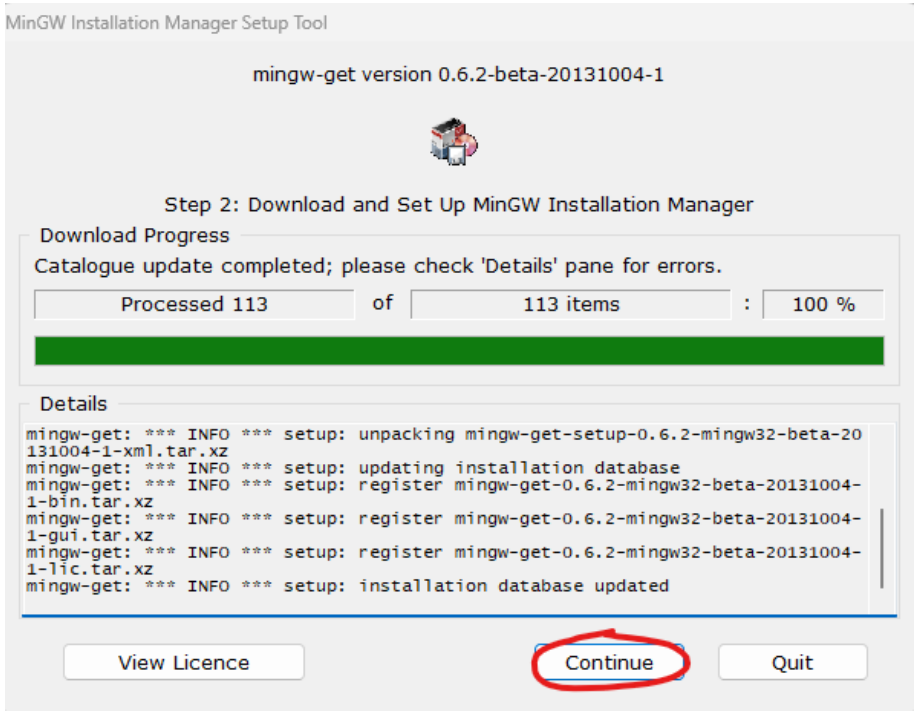
Instalar o MinGW
(https://sourceforge.net/projects/mingw/)

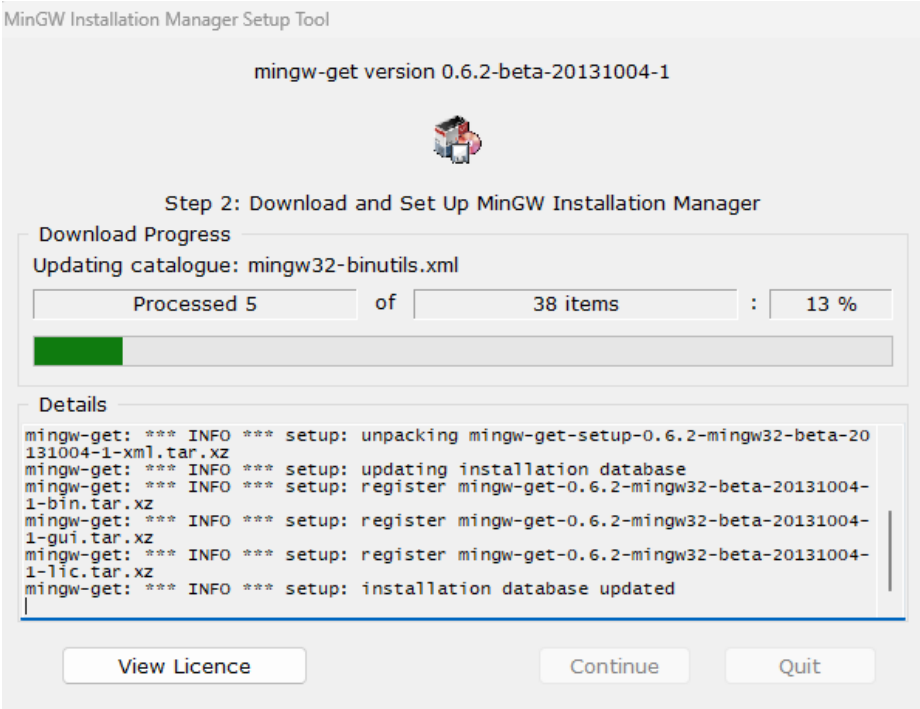
Configurando o WINDOWS (Parte 2)



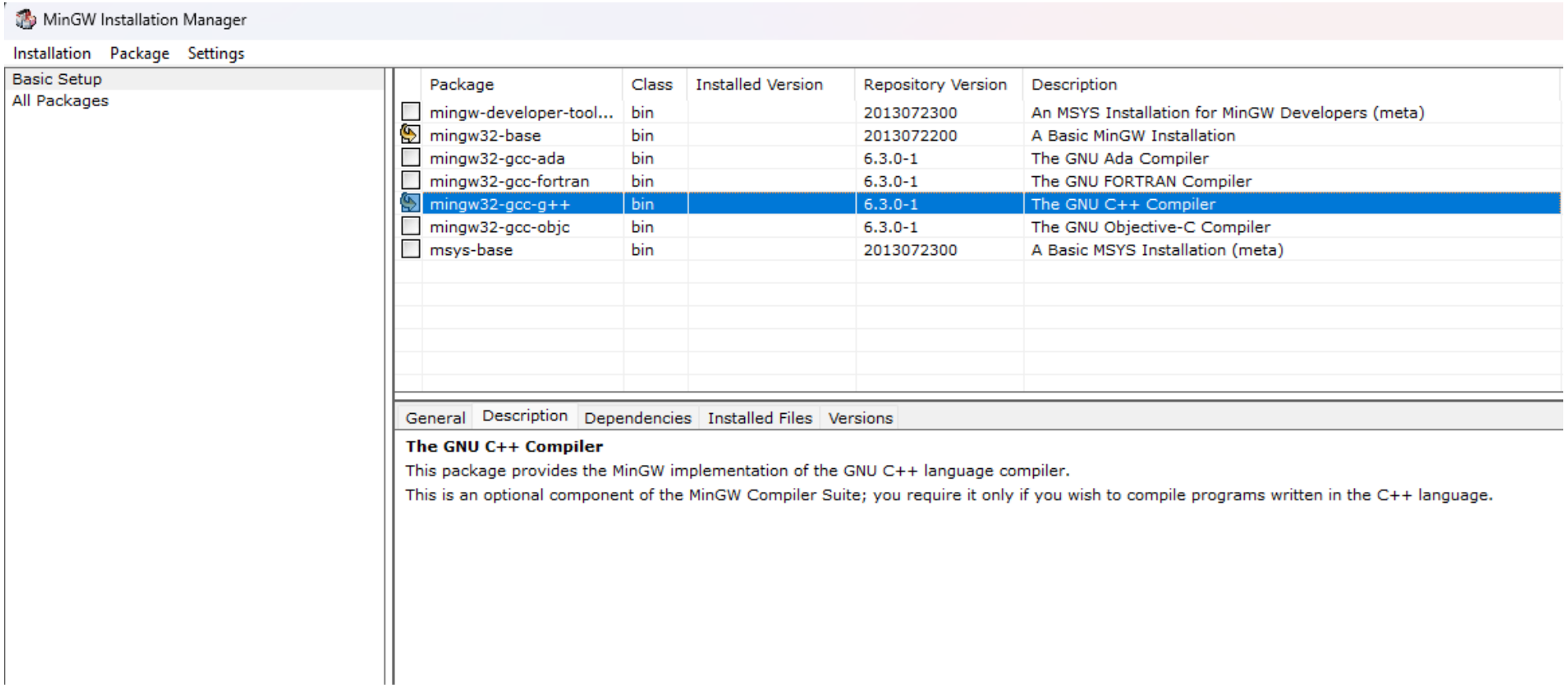


Configurando o WINDOWS (Parte 3)

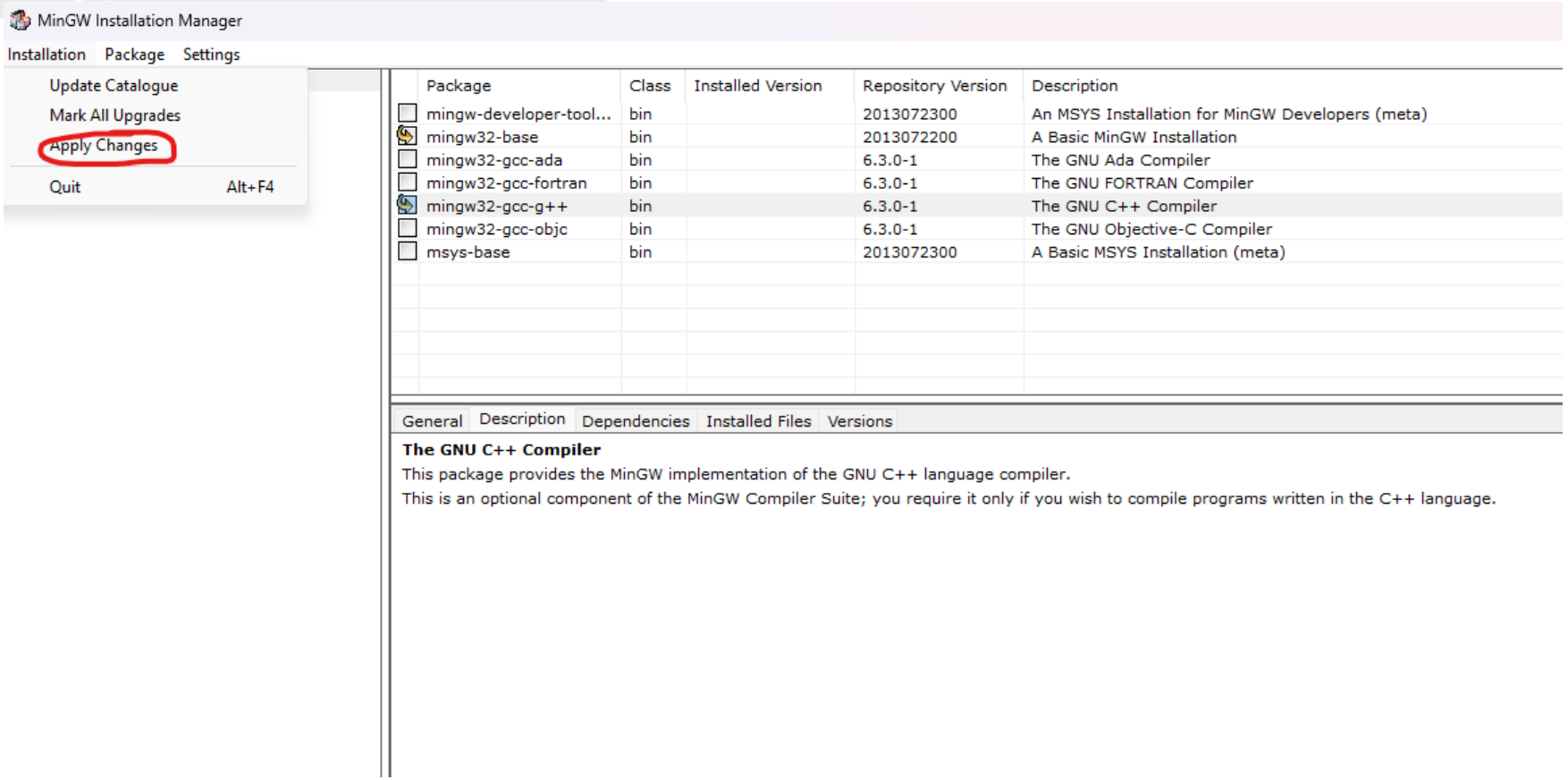




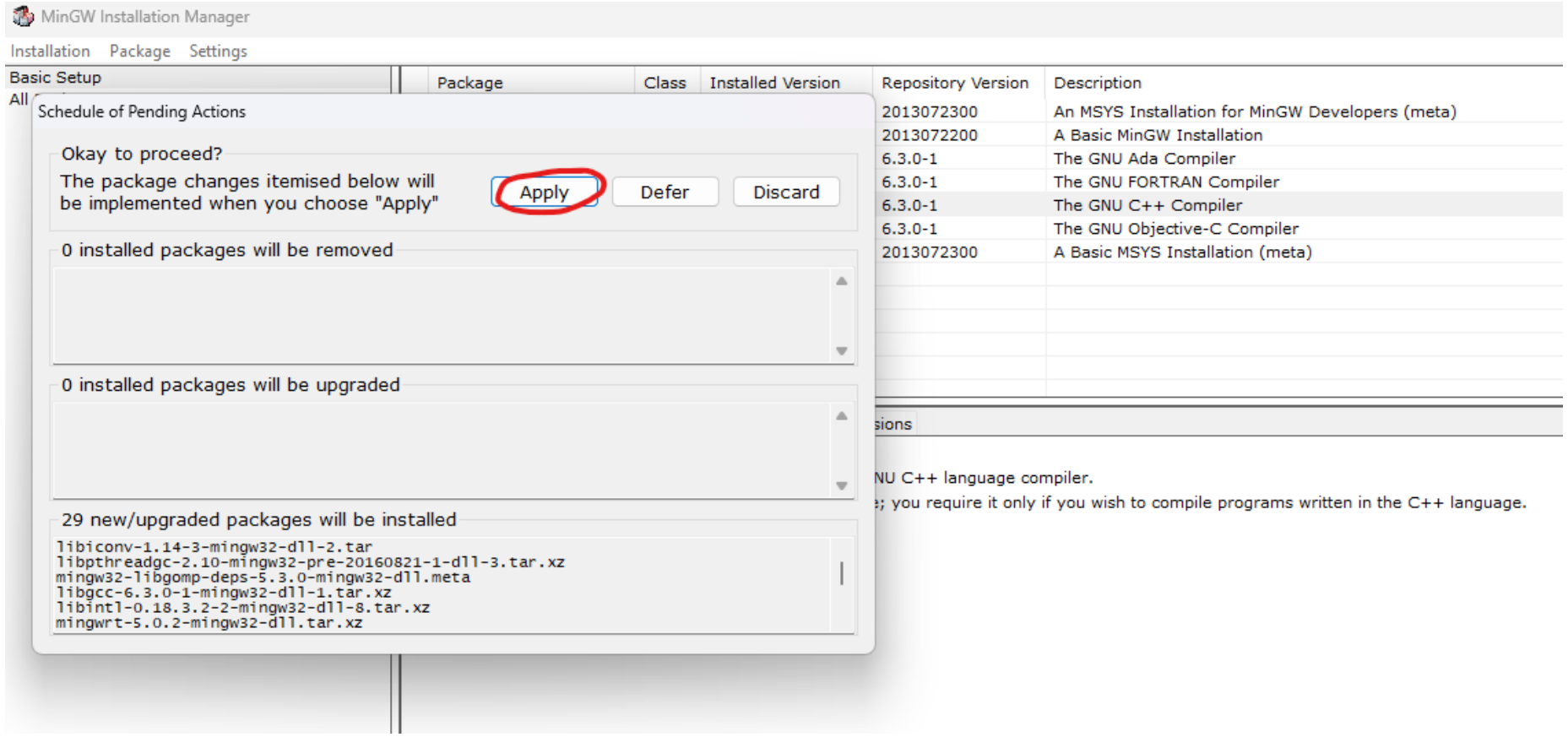
Configurando o WINDOWS (Parte 4)



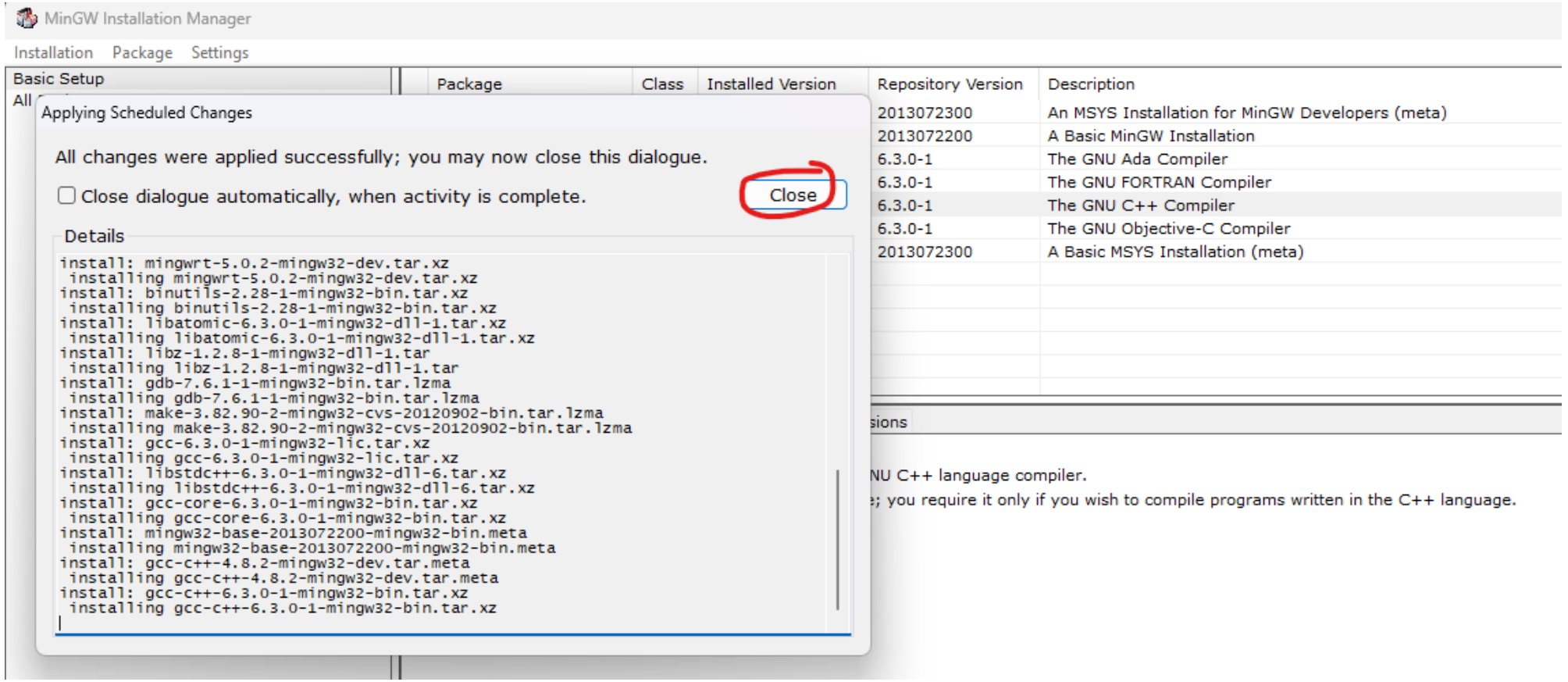
Configurando o WINDOWS (Parte 5)



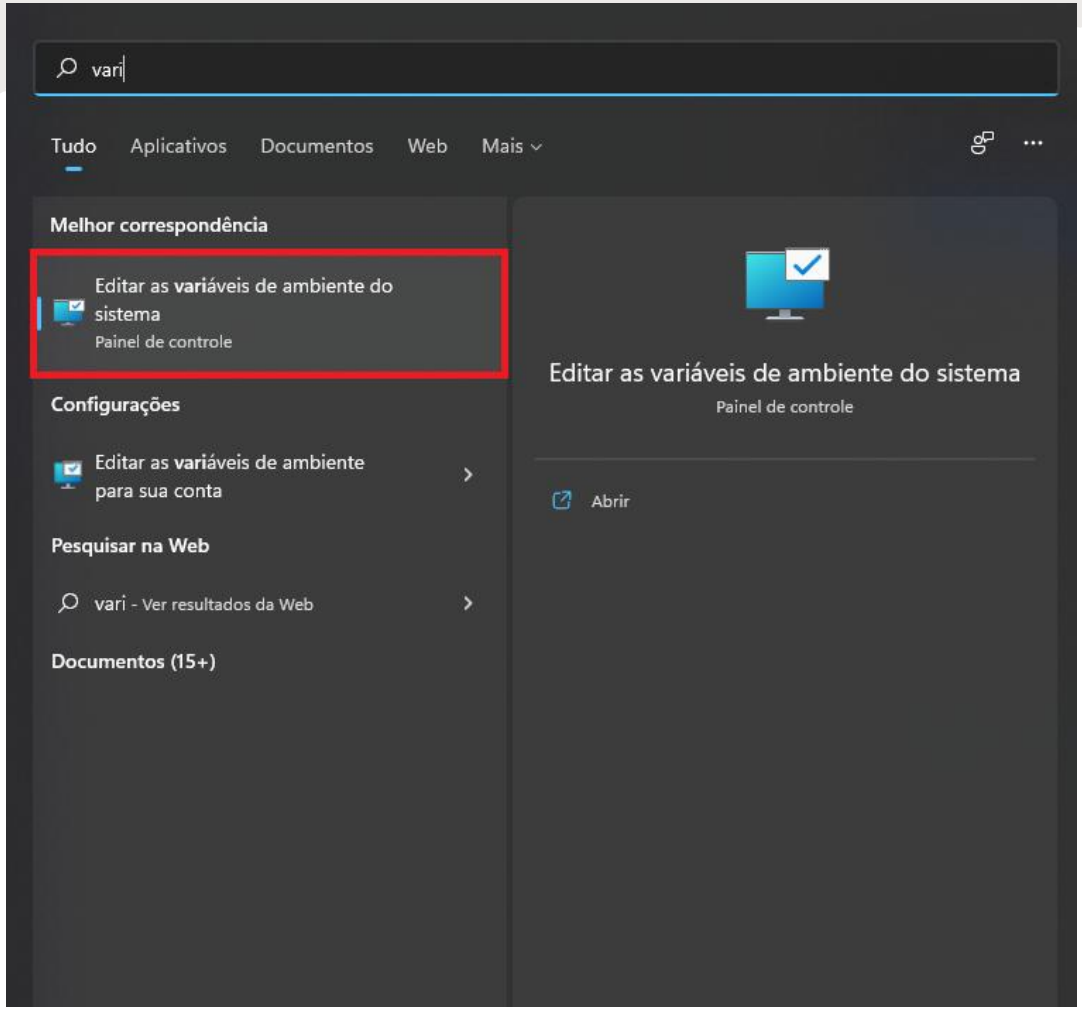
Configurando o WINDOWS (Parte 6)

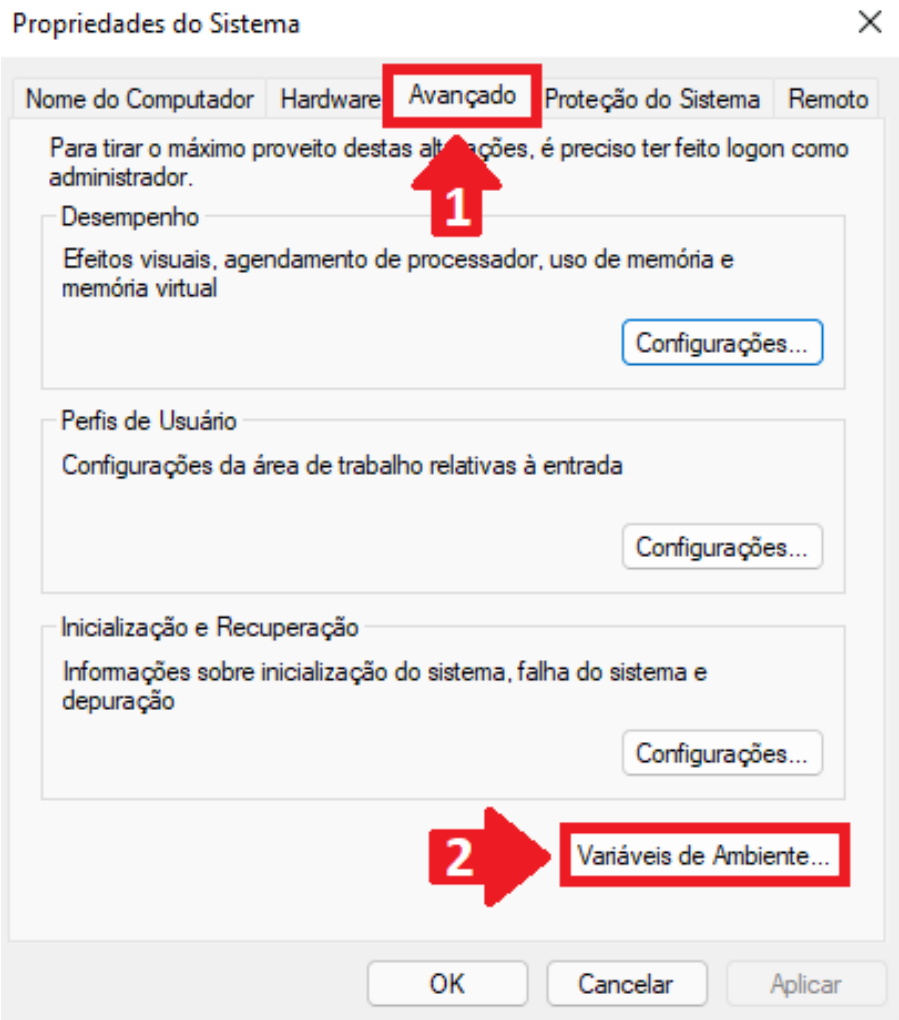


Configurando o WINDOWS (Parte 7)

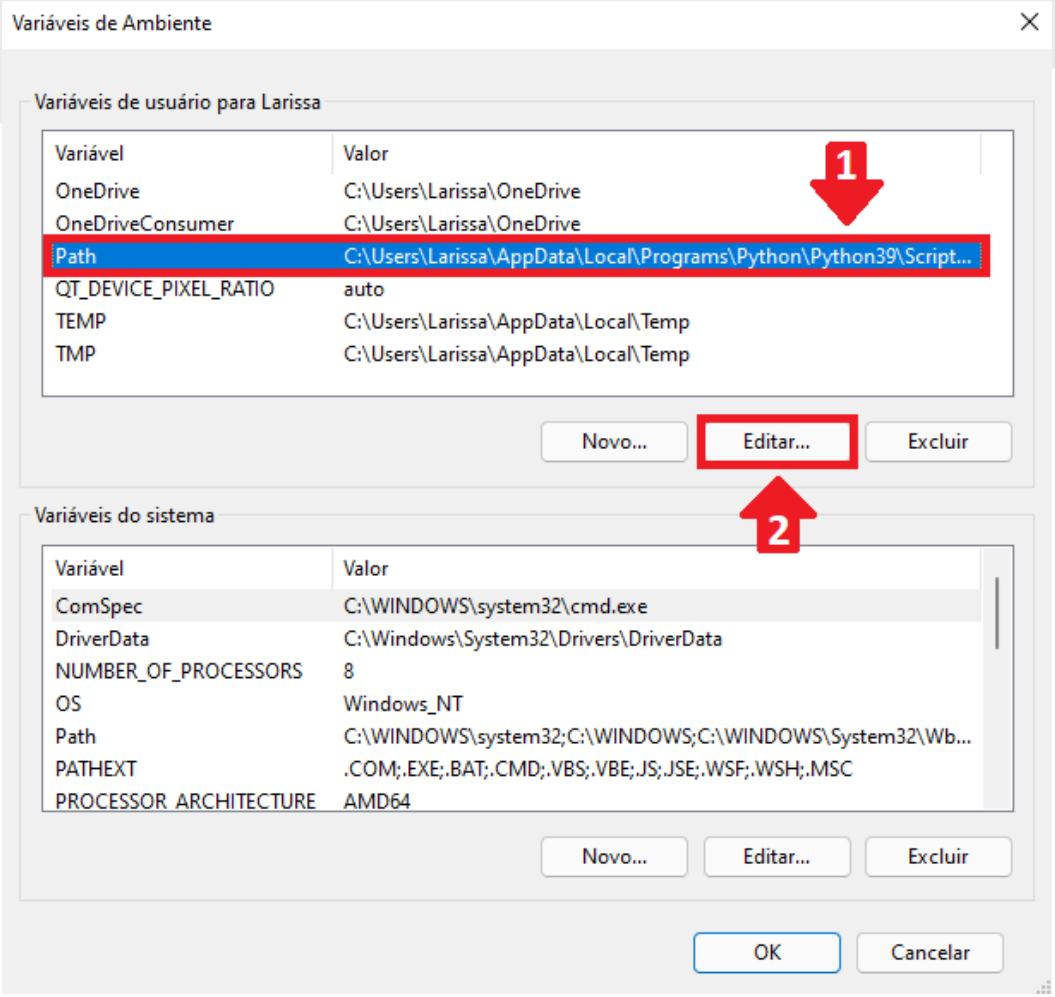


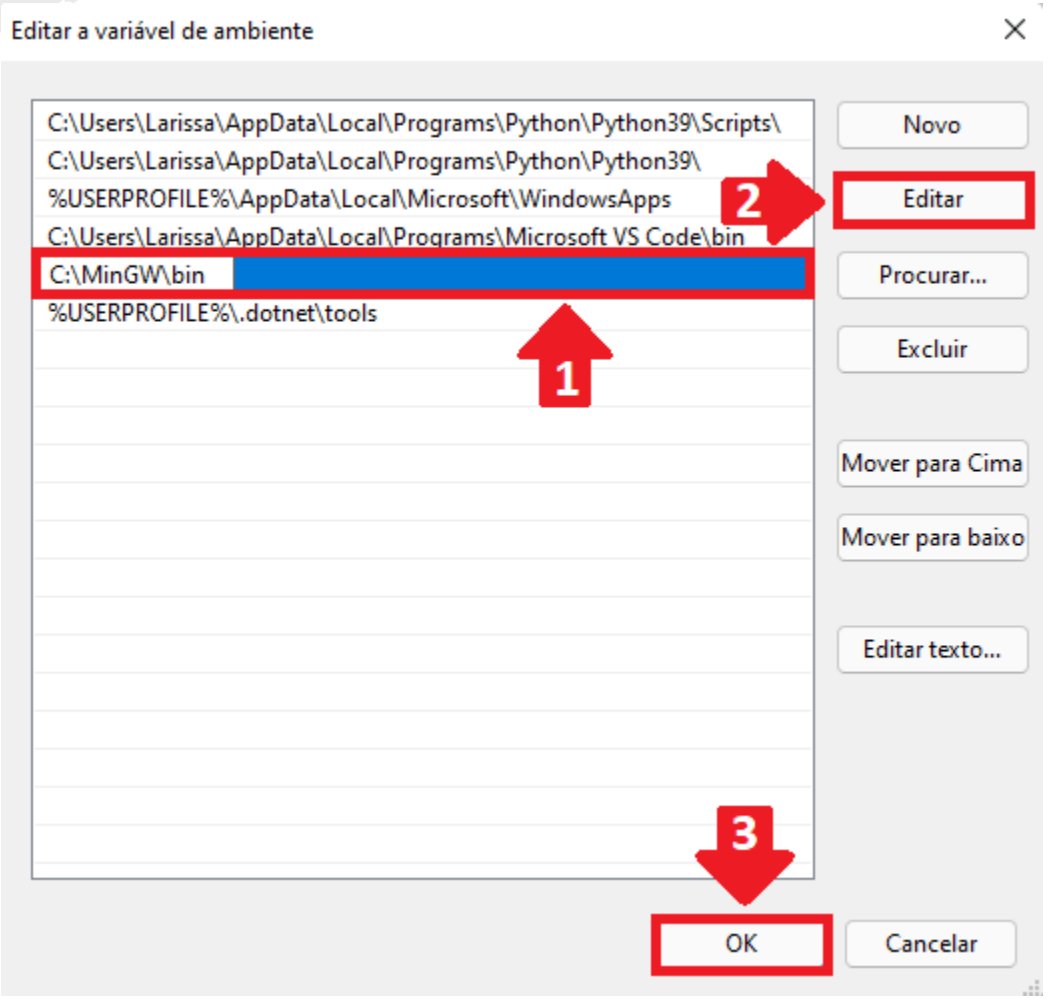
Configurando o WINDOWS (Parte 8)





Configurando o WINDOWS (Parte 9)





Configurando o WINDOWS (Parte 10)

Para abrir o terminal:

Windows + R

Digite “cmd”

Para testar se o gcc está instalado:

$gcc --version

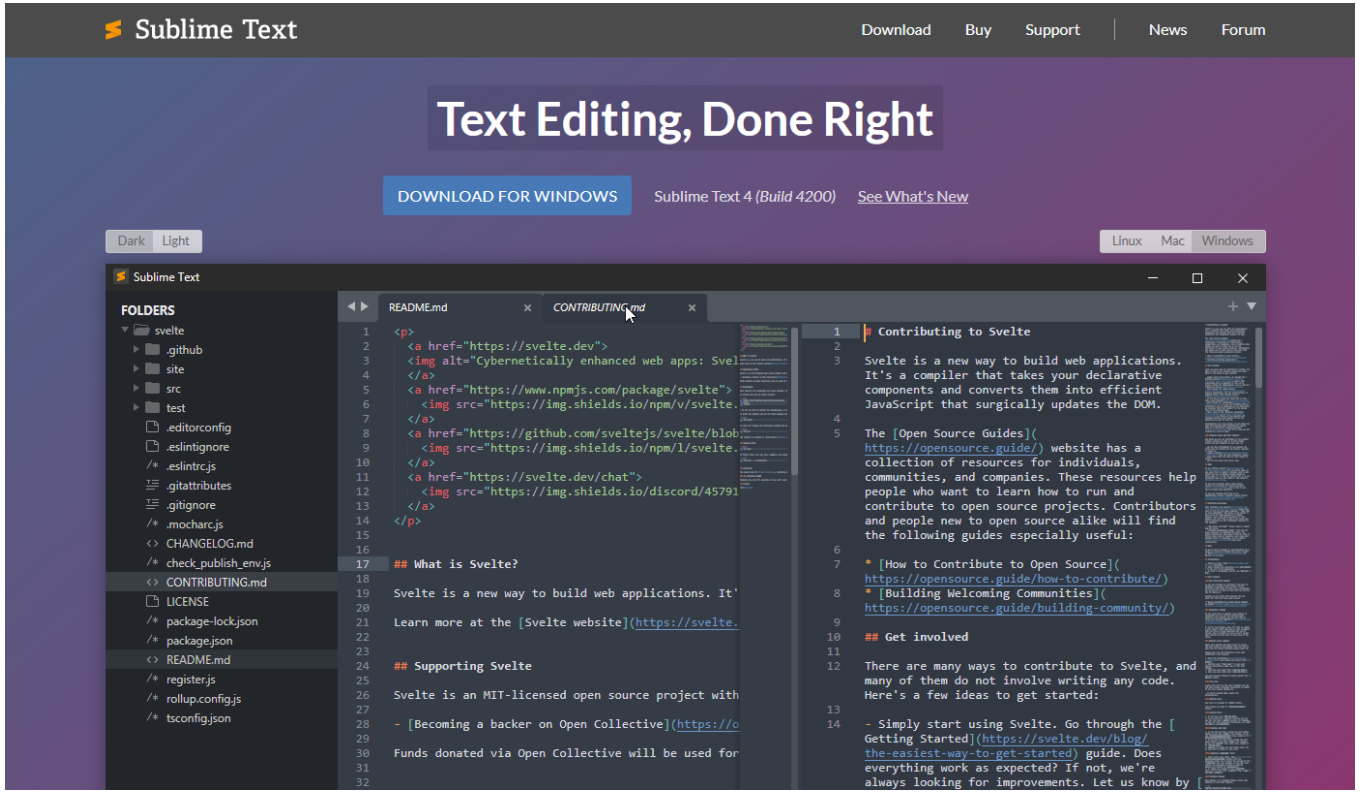
gcc (MinGW.org GCC-6.3.0-1) 6.3.0

Sublime Text no Windows

https://www.sublimetext.com/

Para compilar:

CTRL+B



Compilador online

https://www.onlinegdb.com/online_c_compiler

Primeiros passos em C

Compilando o código

Declarando variáveis

Estruturas condicionais (If, else)

main.c (ver 1) – Olá mundo!

```
#include <stdio.h>

int main() {
        printf(“Ola, mundo!\n");
        return 0;
}
```

OBS: Para compilar:

$gcc main.c –o exe

$./exe

Exercício 1 ~ 5min

Modificar o programa para imprimir 3 frases (em ordem):

“Olá, IFB!”

“Olá, Brasília!”

“Olá, Mundo!”

main.c (ver 2) – Incluindo comentários

```
#include <stdio.h>


int main() {
	// Este é um comentário simples.
	printf("Ola, mundo!\n");
	return 0;
	/* Este é um comentário de
		multiplas
		linhas...
	*/
}
```

Exercício 2 ~ 5min

Adicione comentários para ocultar partes do código.

Identifique erros de compilação.

main.c (ver 3) – Declarando variável inteira e imprimindo na tela

```
#include <stdio.h>

int main() {
	// Este é um comentário!
	int A = 10;
	int B = 20;
	int C = 30;
	// O %d faz referência a variáveis do tipo inteiro.
	printf("O valor da variavel <A> e: %d\n", A);
	return 0;
}
```

OBS: A linguagem C possui “=” como operador de atribuição. No exemplo, a variável “A” está recebendo o valor “10”, a variável “B” está recebendo o valor “20” e assim por diante.

Exercício 3 ~ 5min

Imprima o valor das 3 variáveis em linhas diferentes.

Imprima o valor das 3 variáveis na mesma linha.

Declare uma quarta variável e imprima.

OBS: Variáveis não podem ter o nome repetido dentro do mesmo escopo.

main.c (ver 4) - Lendo informações do teclado

```
#include <stdio.h>

int main() {
    int A;
    printf("Indique um valor: \n");
    scanf("%d", &A);
    return 0;
}
```

OBS: Observe que é necessário adicionar o “&” antes do nome da variável para leitura.

Exercício 4 ~ 5min

Leia 2 valores, armazene em 2 variáveis.

Imprima os 2 valores lidos.

Tipos Inteiros em C

Tipo	Forma Completa	Tamanho Comum	Faixa de Valores (aprox.)
`char`	`char`	1 byte	-128 a 127 ou 0 a 255
`signed char`	`signed char`	1 byte	-128 a 127
`unsigned char`	`unsigned char`	1 byte	0 a 255
`short`	`short int`	2 bytes	-32.768 a 32.767
`unsigned short`	`unsigned short int`	2 bytes	0 a 65.535
`int`	`int`	4 bytes	-2.147.483.648 a 2.147.483.647
`unsigned`	`unsigned int`	4 bytes	0 a 4.294.967.295
`long`	`long int`	4 ou 8 bytes	Depende da arquitetura
`unsigned long`	`unsigned long int`	4 ou 8 bytes	Depende da arquitetura
`long long`	`long long int`	8 bytes	±9 quintilhões aprox.
`unsigned long long`	`unsigned long long int`	8 bytes	0 a ~18 quintilhões

Tipos de Ponto Flutuante em C

Tipo	Forma Completa	Tamanho Comum	Precisão Aproximada
`float`	`float`	4 bytes	~6 a 7 casas decimais
`double`	`double`	8 bytes	~15 casas decimais
`long double`	`long double`	8, 12 ou 16 bytes	Maior que `double`

Exercício 5 ~ 10min

Considerando os padrões para printf e scanf (descritos na tabela ao lado), faça testes de leitura e escrita para:

1. int (%d)
2. long (%ld)
3. float (%f)
4. double (%lf)
5. shot (%hd)

Tipo	printf	scanf
signed char	%hhd	%hhd
unsigned char	%hhu	%hhu
short	%hd	%hd
unsigned short	%hu	%hu
int	%d ou %i	%d
unsigned int	%u	%u
long	%ld	%ld
unsigned long	%lu	%lu
long long	%lld	%lld
unsigned long long	%llu	%llu
float	%f	%f
double	%f	%lf
long double	%Lf	%Lf

main.c (ver 5) - Operações matemáticas

```
#include <stdio.h>

int main() {
	int x, y;
	printf("Primeiro valor: ");
	scanf("%d", &x);

	printf("Segundo valor: ");
	scanf("%d", &y);

	int m = x * y;
	printf("A multiplicacao de %d por %d eh igual a %d\n", x, y, m);
}
```

OBS: É possível declarar duas (ou mais) variáveis do mesmo tipo na mesma linha.

Exercício 6 ~ 10min

Escreva um programa que peça para o usuário digitar duas variáveis inteiras (x e y) e imprima o resultado da:

- Soma (+)
- Subtração (-)
- Multiplicação (*)
- Divisão (/)

OBS: Para divisão, é necessário usar variáveis de ponto flutuante.

main.c (ver 6) - Condicional “if” e “else”

```
#include <stdio.h>

int main() {
        int x;
        printf("Digite um numero: ");
        scanf("%d", &x);
        if (x >= 10) {
                printf("Numero maior ou igual a 10. \n");
        } else {
                printf("Numero menor que 10! \n");
        }
}
```

OBS: A linguagem C aceita diversos operadores relacionais como: > (maior), < (menor), == (igual), != diferente.

Exercício 7 ~ 10min

Leia 2 números. Se a multiplicação deles for maior que 100, imprima “Maior que 100”. Se for menor, imprima “Menor que 100”. Teste outros operadores relacionais:

Operador	Significado	Exemplo
`==`	Igual	`a == b`
`!=`	Diferente	`a != b`
`>`	Maior que	`a > b`
`<`	Menor que	`a < b`
`>=`	Maior ou igual	`a >= b`
`<=`	Menor ou igual	`a <= b`

main.c (ver 7) – Alinhamento de condicionais

```
#include <stdio.h>

int main() {
        int x;
        printf("Digite um numero: ");
        scanf("%d", &x);
        if (x >= 100) {
                printf("Numero maior ou igual a 100! \n");
        } else if (x >= 50) {
                printf("Numero maior ou igual que 50 e menor que 100! \n");
        } else {
                printf("Numero menor que 50! \n");
        }
}
```

Exercício 8 ~ 10min

Faça um programa que receba um número inteiro de 0 a 100. Se for menor do que 0 ou maior que 100, deve imprimir mensagem de erro para cada caso.

main.c (ver 8) – Alinhamento alternativo de condicionais

```
#include <stdio.h>

int main() {
        int x;
        printf("Digite um numero: ");
        scanf("%d", &x);
        if (x >= 100) {
                printf("Numero maior ou igual a 100! \n");
        } else {
            if (x >= 50) {
                printf("Numero maior ou igual que 50 e menor que 100! \n");
            } else {
                printf("Numero menor que 50! \n");
            }
        }
}
```

Exercício 9 ~ 10min

O que faz o código que segue?

```
#include <stdio.h>
int main() {
        int chute = 0;
        int numerosecreto = 42;
        int acertou = chute == numerosecreto;
        if(acertou) {
              printf("Parabéns! Você acertou!\n");
        } else {
            if(chute > numerosecreto) {
                        printf("Seu chute foi maior do que o número secreto!\n");
            }
            if(chute < numerosecreto) {
                        printf("Seu chute foi menor do que o número secreto!\n");
            }
        }
}
```

Exercício 10 ~ 10min

Modifique o código anterior para ler o “chute” do usuário. O usuário pode ter 3 tentativas para acertar o “número secreto”

AULA 03

Condicionais: if, else
Estruturas de repetição: for
Outros: break, continue

main.c (ver 9) – Escopo de uma variável

```
#include <stdio.h>

int main() {
        int x = 100;
        if (x > 10) {
                int y = 10;
                printf("O valor de y: %d \n", y);
        } else {
                 printf("Numero menor que 10! \n");
        }
}
```

Exercício 11 ~ 5min

O código a seguir funciona? Como corrigir ele?

```
#include <stdio.h>
int main() {
	if(100 > 20) {
		int variavel = 1;
		variavel = variavel + 1;
	}
	variavel = variavel + 1;
}
```

main.c (ver 10) – Estrutura de repetição “for”

```
#include <stdio.h>

int main() {
        int i = 100;
        for (int i = 0; i < 100; i++) {
                printf("%d ", i);
        }
}
```

OBS: O “i++” é um operador de incremento, igual a “i=i+1”.

Exercício 12 ~ 5min

A partir do código anterior, altere o for para utilizar o operador de decremento “i--”.

Operadores de Incremento e Decremento

i++ : usa o valor, depois incrementa.

++i : incrementa, depois usa o valor.

Exemplo:

- int i = 3;
- int resultado = i++ + 5;
- ou
- int i = 3;
- int resultado = ++i + 5;

Exercício 13 ~ 5min

Faça um programa que peça um número inteiro ao usuário e imprima a tabuada desse número de 1 a 10.

Exemplo (entrada = 5):

5 x 1 = 5

5 x 2 = 10

...

5 x 10 = 50

Exercício 14 ~ 5min

Faça um programa que peça um número inteiro positivo N e calcule a soma de todos os números de 1 até N usando for.

Exercício 15 ~ 5min

Faça um programa que:

- Leia 10 números inteiros.
- Conte quantos são pares e quantos são ímpares.
- Mostre o resultado ao final.

Dica: Use o operador %: Se “X % 2 == 0”, então X é divisível por 2.

main.c (ver 11) - Encerrando a estrutura de repetição com “break” e o uso de “define”

```
#include <stdio.h>
#define MAX 10
#define CHAVE 4

int main() {
	for (int i = 0; i < MAX; i++) {
		printf("%d ", i);
		if (i == CHAVE) break;
	}
}
```

OBS: O “define” é utilizado para definição de constantes no código. O “break” encerra abruptamente a estrutura de repetição.

Exercício 16 ~ 15min

Faça um programa em C que:

- Peça ao usuário para digitar 10 números inteiros.
- Use um laço for para ler os valores.
- Se o usuário digitar o número 0, o programa deve:
  - Interromper o laço imediatamente usando break
  - Informar em qual posição o número 0 foi digitado

Ao final, o programa deve mostrar:

- Quantos números foram digitados antes da interrupção
- A soma dos números digitados (desconsiderando o zero)

main.c (ver 12) – Uso de “continue”

```
#include <stdio.h>
#define MAX 10
#define CHAVE 4

int main() {
        for (int i = 0; i < MAX; i++) {
                if (i == CHAVE) continue;
        printf("%d ", i);
        }
}
```

OBS: O “continue” pula o restante do “for” e vai para a próxima iteração.

Exercício 17 ~ 15min

Faça um programa que:

- Leia 10 números inteiros.
- Some apenas os números pares.
- Ignore os números ímpares utilizando a instrução continue.
- Mostre a soma final.

Exercício 18 ~ 30min

Faça um programa que:

- Leia um número inteiro positivo.
- Verifique se ele é primo.
- Exiba uma mensagem informando o resultado.

OBS: Um número é primo se é divisível apenas por 1 e ele mesmo.

Exercício 19 ~ 30min

Faça um programa que:

- Leia 8 números inteiros.
- Verifique se algum número digitado é negativo.
- Se encontrar um número negativo:
  - Exiba a mensagem: "Número inválido digitado."
  - Interrompa o laço imediatamente usando break.

Exercício 20 ~ 30min

Faça um programa que:

- Peça ao usuário um número inteiro N.
- Mostre todos os números de 1 até N.

Porém:

- Ignore os múltiplos de 3 usando continue.
- Se encontrar um número maior que 50, interrompa o laço usando break.

Para cada número exibido:

- Se for par, escreva "PAR".
- Se for ímpar, escreva "ÍMPAR".

AULA 04

Estruturas de repetição: while
Operações matemáticas com casas decimais
Convertendo tipos de variáveis
Arredondamento de valores

main.c (ver 13) - Estrutura de repetição “while”

```
#include <stdio.h>

int main() {
    int contador = 1;
    while (contador <= 5) {
        printf("%d\n", contador);
        contador++;  // incrementa o contador
    }
    return 0;
}
```

OBS: A estrutura “while” irá continuar enquanto a condição entre parênteses for verdadeira.

Exercício 21 ~ 10min

Faça um programa que imprima todos os números de 1 a 100 usando o laço de repetição “while”.

Exercício 22 ~ 30min

Escreva um programa que imprima a soma de todos os números de 1 até 100. Ou seja, ele calculará o resultado de 1+2+3+4+...+100.

Exercício 23 ~ 30min

Faça um programa que:

- Leia números inteiros digitados pelo usuário.
- Continue lendo enquanto o número for positivo.
- Quando o usuário digitar um número negativo, o programa deve parar.

Ao final, exiba:

- A quantidade de números digitados (positivos).
- A soma desses números.
- A média deles.

main.c (ver 14) - Trabalhando com casas decimais

```
#include <stdio.h>

int main() {
        int nota;
        float soma = 0.0;
        for (int i=0; i < 3; i++) {
            printf("Informa a Nota %d: ", i);
            scanf("%d", &nota);
            soma += nota;
        }
        printf("\nA nota média é %.2f", soma/3);
        return 0;
}
```

OBS: O “%.2f” indica que será impresso um float com 2 casas decimais.

Exercício 24 ~ 10min

Crie um programa que:

- Leia **3 notas decimais** (tipo float).
- Calcule a média aritmética.
- Mostre o resultado com **2 casas decimais.**

Exercício 25 ~ 10min

Crie um programa que:

- Leia uma temperatura em **Celsius** (tipo double).
- Converta para **Fahrenheit.**

Fórmula:

- $F = (C \times 9/5) + 32$

Exercício 26 ~ 10min

Crie um programa que:

- Leia:
  - Capital inicial (double)
  - Taxa de juros (%) (float)
  - Tempo em meses (int)

Calcule o **montante final** usando juros simples:

- $M = C \times (1 + (taxa/100 \times tempo))$

main.c (ver 15) – Convertendo tipos de variáveis (casting)

```
#include <stdio.h>

int main() {
        int a = 5, b = 2;
        float f1, f2;
        f1 = a/b;
        f2 = (float) a/b;

        printf("O valor de <f1> e: %.1f \n", f1);
        printf("O valor de <f2> e: %.1f \n", f2);
        return 0;
}
```

Exercício 27 ~ 10min

Crie um programa que:

- Leia dois números inteiros
- Calcule a média
- Garanta que o resultado seja decimal usando casting
- Sem casting, a média pode sair errada (divisão inteira).

Exercício 28 ~ 10min

Crie um programa que:

- Declare: double valor = 9.78;
- Converta explicitamente esse valor para int
- Mostre os dois valores

Perguntas:

- O que acontece com a parte decimal?
- Houve arredondamento ou truncamento?

main.c (ver 16) – Funções de arredondamento

```
#include <stdio.h>
#include <math.h>
int main() {
        int a, b, c;
        // Arredonda para cima.
        a = ceil(4.5);
        // Arredonda para baixo.
        b = floor(4.5);
        // Arredondamento tradicional.
        c = round(4.5);
        printf("O valor de <a> e: %d \n", a);
        printf("O valor de <b> e: %d \n", b);
        printf("O valor de <c> e: %d \n", c);
        return 0;
}
```

Exercício 29 ~ 10min

Crie um programa que:

- Leia um número decimal (double)
- Mostre:
  - Valor original
  - Valor usando ceil()
  - Valor usando floor()
  - Valor usando round()

Exercício 30 ~ 10min

Crie um programa que:

- Leia três notas decimais (double)
- Calcule a média
- Arredonde a média usando:
  - floor() (critério rígido)
  - ceil() (critério generoso)
  - round() (critério padrão)

Depois informe:

- Se o aluno está aprovado (nota final ≥ 6)

AULA 05

Números randômicos
Estrutura switch

main.c (ver 17) – Gerando números pseudo-randômicos

```
#include <stdio.h>
#include <stdlib.h>

int main() {
        int a, b, c;
        // Gera um numero randomico de 0-100
        a = rand() % 100;
        b = rand() % 100;
        c = rand() % 100;

        printf("O valor de <a> e: %d \n", a);
        printf("O valor de <b> e: %d \n", b);
        printf("O valor de <c> e: %d \n", c);
        return 0;
}
```

main.c (ver 18) – Gerando números randômicos com semente

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
        int a, b, c;
        int tempo = time(0);
        srand(tempo);
        // Gera um numero randomico de 0-100
        a = rand() % 100;
        b = rand() % 100;
        c = rand() % 100;
        printf("O valor de <a> e: %d \n", a);
        printf("O valor de <b> e: %d \n", b);
        printf("O valor de <c> e: %d \n", c);
        return 0;
}
```

Exercício 31 ~ 10min

Crie um programa que:

- Leia dois números inteiros (mínimo e máximo)
- Gere um número aleatório dentro desse intervalo (inclusive)

Fórmula geral: min + rand() % (max - min + 1)

Exercício 32 ~ 10min

Crie um programa que:

- Simule o lançamento de um dado (valores de 1 a 6)
- Mostre o resultado
- Pergunte ao usuário se deseja lançar novamente (usar while)

Exercício 33 ~ 10min

Crie um programa que:

- Gere 100 números aleatórios entre 0 e 4
- Conte quantas vezes cada número apareceu
- Mostre a frequência de cada valor

Exercício 34 ~ 10min

Crie um programa que:

- Gere um número aleatório entre 1 e 100
- Peça para o usuário tentar adivinhar
- Informe:
  - “Muito alto”
  - “Muito baixo”
  - “Acertou!”
- Conte o número de tentativas

main.c (ver 19) - Uso da estrutura switch

```
#include <stdio.h>
int main() {
        int nivel;
        printf("Escreva o nivel do jogo: \n(1) Facil (2) Medio (3) Dificil\n"); scanf("%d", &nivel);
        switch (nivel) {
            case 1:
                printf("Nivel Facil escolhido");
                break;
            case 2:
                printf("Nivel Medio escolhido");
                break;
            case 3:
                printf("Nivel Dificil escolhido");
                break;
            default:
                printf("Opcao invalida!");
        }
        return 0;
}
```

Exercício 35 ~ 10min

Faça um programa que leia um número de 1 a 7 e imprima o dia da semana correspondente.

Exemplo:

1 → Domingo

2 → Segunda-feira

...

7 → Sábado

Exercício 36 ~ 10min

Faça um programa que:

- Leia dois números float
- Leia um caractere representando a operação: +, -, *, /
- Use switch para realizar a operação escolhida.
- Trate divisão por zero.

Exemplo:

- Digite: 10 5 *
- Resultado: 50

Exercício 37 ~ 10min

Crie um menu que exiba:

- 1 - Cadastrar usuário
- 2 - Listar usuários
- 3 - Excluir usuário
- 4 - Sair

O programa deve:

- Ler a opção
- Usar switch para mostrar uma mensagem correspondente
- Se for 4, encerrar o programa
- Caso contrário, mostrar "Opção inválida" se necessário

Exercício 38 ~ 10min

Peça ao usuário uma nota de 0 a 10.

Use switch para classificar:

- 9 e 10 → "Excelente"
- 7 e 8 → "Bom"
- 5 e 6 → "Regular"
- 0 a 4 → "Reprovado"

(Dica: use vários case seguidos para o mesmo bloco.)

Exercício 39 ~ 10min

Faça um programa que leia o número da porta e informe o protocolo TCP mais comum associado:

- 80 → HTTP
- 443 → HTTPS
- 21 → FTP
- 22 → SSH
- 25 → SMTP

Caso não esteja na lista, mostrar:

- Porta não cadastrada no sistema.

Exercício 40 ~ 10min

Faça um programa que:

Leia 12 números inteiros.

- Considere como válidos apenas os números entre 10 e 50 (inclusive).
- Se o número estiver fora desse intervalo:
  - Ignore-o usando continue.
- Para os números válidos:
  - Se forem pares, some ao total de pares.
  - Se forem ímpares, some ao total de ímpares.

Ao final, mostre:

- Quantidade de números válidos
- Soma dos pares válidos
- Soma dos ímpares válidos

AULA 06

Declarando arrays
Percorrendo e varrendo arrays

main.c (ver 20) - Criando arrays básicos

```
#include <stdio.h>

int main() {
        int notas[3];
        // Observe que a primeira posição é 0.
        notas[0] = 7;
        notas[1] = 4;
        notas[2] = 8;

        float res = (notas[0] + notas[1]+ notas[2])/3;
        printf("A média de notas é: %.2f \n", res);
        return 0;
}
```

Exercício 41 ~ 10min (V2)

Crie um programa que:

- Declare um vetor de 10 números inteiros.
- Peça ao usuário para preencher o vetor.
- Determine:
  - Maior valor
  - Menor valor
  - Média aritmética
- Exiba os resultados.

Exercício 42 ~ 10min

Faça um programa em C que declare um vetor de 15 números inteiros e leia seus valores.

Depois, o programa deve informar:

- quantos números pares existem no vetor;
- quantos números ímpares existem no vetor;

Exercício 43 ~ 10min

Declare dois vetores de 10 posições com números inteiros lidos pelo usuário.

Crie um terceiro vetor contendo apenas os valores que aparecem nos **dois vetores**, sem repetir elementos. Ao final, exiba o vetor interseção.

Exemplo:

Vetor A: 1 2 3 4 5 6 7 8 9 10
Vetor B: 5 7 11 13 2 20 8 30 1 40
Interseção: 1 2 5 7 8

Exercício 44 ~ 10min

Faça um programa que leia um vetor de 10 inteiros e um número k. O programa deve rotacionar os elementos do vetor k posições para a direita.

Exemplo:

Vetor original: 1 2 3 4 5 6 7 8 9 10

k = 3

Resultado: 8 9 10 1 2 3 4 5 6 7

Considere que k é menor que o tamanho do vetor.

main.c (ver 21) – Array de caracteres

```
#include <stdio.h>

int main() {
    char nomed[40];
    sprintf(nomed, "Algoritmos e Programacao de Computadores");
    printf("%s \n", nomed);
    // Para encerrar a string, adicionar o '\0'
    nomed[30] = '\0';
    printf("%s \n", nomed);
    return 0;
}
```

OBS: A função “sprintf()” escreve uma frase dentro de uma variável do tipo char [].

main.c (ver 22) – Função strlen()

```
#include <stdio.h>
#include <string.h>

int main() {
        char frase[30] = "O homem e o lobo do homem";
        for (int i=0; i < strlen(frase); i++) {
            printf("%c ", frase[i]);
        }
        return 0;
}
```

OBS: A função “strlen()” retorna o tamanho da string.

main.c (ver 23) – Leitura de strings

```
#include <stdio.h>
#include <string.h>

int main() {
        char nome[30];
        printf("Digite o seu nome: ");
  // Le 30 caracteres da entrada padrao: teclado.
        fgets(nome,30, stdin);
  // strcspn procura o caractere "\n" e retorna o indice.
        nome[strcspn(nome, "\n")] = '\0';
        printf("\nNome informado: <%s>", nome);
        return 0;
}
```

Exercício 45 ~ 10min

Leia uma string (com espaços).

- Conte quantas vogais existem.
- Mostre o total.
- Dica: usar fgets().

Exercício 46 ~ 10min

Leia uma palavra.

- Verifique se ela é um palíndromo.
- Exiba se é ou não.

Exemplo:

- ovo → é palíndromo
- casa → não é

Exercício 47 ~ 10min

Leia uma frase.

- Crie outra string sem os espaços.
- Mostre o resultado.

Exercício 48 ~ 10min

Leia dois vetores de 5 posições.

Crie um terceiro vetor intercalando os valores.

Exemplo:

- A = {1,2,3}
- B = {4,5,6}
- Resultado = {1,4,2,5,3,6}

Exercício 49 ~ 10min

Leia uma string.

- Conte quantas vezes cada letra do alfabeto aparece.
- Ignore maiúsculas/minúsculas (trate como iguais).
- Mostre apenas as letras que aparecem.

Exemplo:

- Entrada: "Ana Maria"
- Saída:

  a: 4

  m: 1

  r: 1

  i: 1

  n: 1

Exercício 50 ~ 10min

Faça um programa que:

- Leia uma frase usando fgets().
- Leia um caractere a ser substituído.
- Leia o novo caractere.
- Percorra a string e substitua todas as ocorrências.
- Mostre a string modificada.

Exemplo:

Frase: banana

Substituir: a

Novo: o

Resultado: bonono

AULA 07

Prova 1 – Aguarde orientação do professor

Principais Bibliotecas da Linguagem C

Biblioteca	Finalidade	Principais Funções	Quando Usar	Observações Importantes	Exemplo
stdio.h	Entrada e saída de dados	`printf()`, `scanf()`, `fopen()`, `fclose()`, `fprintf()`, `fscanf()`	Exibir mensagens, ler dados, manipular arquivos	Essencial para quase todo programa iniciante	`printf("Olá");`
stdlib.h	Funções utilitárias e controle de memória	`malloc()`, `free()`, `exit()`, `rand()`, `atoi()`, `system()`	Alocação dinâmica, números aleatórios, conversões	Fundamental para estruturas dinâmicas (listas, pilhas etc.)	`int *p = malloc(sizeof(int));`
string.h	Manipulação de strings (char[])	`strlen()`, `strcpy()`, `strcmp()`, `strcat()`, `strncpy()`	Trabalhar com textos	Strings em C são vetores de `char`, não existe tipo `string` nativo	`strlen(nome);`
math.h	Funções matemáticas	`sqrt()`, `pow()`, `sin()`, `cos()`, `log()`	Cálculos científicos ou matemáticos	Precisa compilar com `-lm` → `gcc prog.c -lm`	`sqrt(9);`
time.h	Manipulação de tempo e data	`time()`, `clock()`, `difftime()`, `localtime()`	Trabalhar com horário do sistema	Muito usada para inicializar `rand()`	`time(NULL);`
ctype.h	Manipulação e teste de caracteres	`isdigit()`, `isalpha()`, `toupper()`, `tolower()`	Validação de entrada de usuário	Muito útil para validar dados digitados	`isdigit('5');`